

Diagnostic : où part le temps

Périmètre. Le chemin de production passe par quatre couches :

- le navigateur (index.html, terrain_lod.js) ;
- Flask avec une voie de calcul unique (terrain_server.py, terrain_jobs.py) ;
- InfiniteTensor ;
- trois UNet EDM PyTorch CUDA BF16 (terrain_inference.py).

Relecture du coordinateur : estimations et scénarios ci-dessous restent à mesurer ; les reçus historiques ne constituent pas une baseline actuelle. L'original et la provenance CLI sont conservés. Le probe webgpu/ (ONNX/WASM) est hors périmètre. Les reçus portent sur une RTX 3090 avec PyTorch 2.11 et CUDA 12.8, sous une charge de bureau non contrôlée. Je n'ai exécuté aucune mesure nouvelle.

Faits mesurés (reçus existants)

- **LOD 4**, seed 0 (progression-final.json) : 0,284 s. 80 forwards coarse en batch 1, soit 0,141 s d'événements CUDA.
- **LOD 3** : 1,923 s. 290 fenêtres base en 19 appels, soit 1,783 s CUDA (93 %).
- **LOD 2** : 2,193 s. 25 decoder 512² en batch 1 = 1,834 s, plus 38 fenêtres base = 0,259 s (≈95 %).
- **benchmark_runtime_lod final** : LOD 3 médiane 2,117 s. LOD 2 varie de 3,69 à 5,66 s selon le run.
- **Navigateur « natural »**, poids résidents :
 - jalons : LOD 4 à 1,11 s, LOD 3 à 3,09 s, LOD 2 à 5,39 s, natif à 7,57 s ;
 - compteurs du monde : 374 fenêtres base, 40 decoder, 4 coarse.
- **Terrestre froid**, seed 42 : couverture native à 12,44 s. 512 fenêtres base et 40 decoder. Des tuiles d'aperçu CPU attendent 1,47 à 1,65 s en file.
- **Coûts ponctuels** :
 - création complète du bootstrap : 1,18 à 2,34 s au premier accès (pas le Rust seul), 0,04 s en relecture ;
 - chargement des modèles : 2,5 à 3,6 s ;
 - revisite de 25 tuiles en cache : 0,54 s.

Déductions

- **Coûts unitaires** : ≈6,1 ms par fenêtre base (batch 16), ≈73 ms par fenêtre decoder, ≈1,8 ms par pas coarse.
- **Part du NN estimée** : reconstruction depuis compteurs et coûts unitaires, sans timeline commune ; sur le parcours naturel, le NN prend ≈5,5 s sur 7,57 s (≈72 %). Il reste ≈2,1 s hors NN : file, CPU par tuile, écritures, barrières.

- **FLOP** (calculés d'après les configs des checkpoints, non mesurés) :
 - base : ≈96 GMAC par fenêtre 64² (192 canaux, multiplicateurs 1-2-3-4, 3 blocs) ;
 - decoder : ≈690 GMAC par tuile 512² (64 canaux) ;
 - les convolutions 3×3 font plus de 90 % des FLOP.
- **Débit effectif** : ≈31 TFLOPS pour le base, ≈19 TFLOPS pour le decoder. Le pic BF16 dense est de l'ordre de 71 TFLOPS, pic constructeur, pas un débit soutenu : [NVIDIA, tableau 7](#).

Amdahl

- **Ce qu'exige un x5 à travail constant** :
 - LOD 3 : le hors-NN fait 0,14 s, donc le NN doit être 7,3× plus rapide ;
 - LOD 2 : le hors-NN fait 0,10 s, donc il faut 6,2×.
- **Planchers théoriques à 100 % du pic** :
 - LOD 3 : ≈0,78 s, au-dessus de la cible x5 de 0,385 s ;
 - LOD 2 : ≈0,59 s, au-dessus de la cible de 0,44 s.
- **Avis : x5 par empreinte paraît peu plausible sous ces hypothèses.** Les GMAC sont non audités ; ces plafonds ne constituent pas une preuve d'impossibilité.
- **Bout en bout** : doubler seulement la vitesse du NN ne donne qu'environ 1,55×.

Limites des preuves

- **Échantillons** : seulement trois sites et cinq répétitions. Les p95 sont interpolés.
- **GPU** : partagé, déjà actif à 26-35 % avant les runs.
- **Événements CUDA** : imbriqués, ils peuvent inclure des trous.
- **Timeline** : aucune timeline serveur par tuile n'existe. Les FLOP et le pic sont non mesurés.
- Les temps des profils Macro A4 historiques ne sont pas attribués au bootstrap terrestre.

Moteur spécialisé aux poids ou compilation ?

Opérations dominantes (déductions depuis `edm_unet.py` et `unet_block.py`)

- **Convolutions 3×3** en NCHW BF16, avec accumulation FP32 par cuDNN : l'essentiel des FLOP. Dans le decoder, les niveaux 512² et 256² côté décodage (blocs up à 128 et 192 canaux, `unet_block.py:116-156`) pèsent ≈55 %. Les convolutions à 64 canaux exploitent mal les tensor cores.

- Opérations limitées par la mémoire :

- `normalize` (`mp_layers.py:9-12`) ;
- `mp_silu` et la modulation par embedding ;
- `mp_sum`, qui matérialise un `torch.stack` (`terrain_nn_constants.py:49`) ;
- `clip` et `mp_concat`.

Estimation : 15 à 25 % du temps base, davantage au decoder.

- **Rééchantillonnage** : des convolutions groupées à poids unitaires (`terrain_nn_constants.py:71-81`) qui cachent un simple découpage ou une répétition.

- **Attention** : seulement au midblock base, sur 8×8 jetons. Avec `image_size` 512, `attn_resolutions` [8,16] ne correspond à aucun niveau. C'est une priorité secondaire ; le gain éventuel reste à profiler.

- **Coarse** : 64², batch 1, 20 pas, ≈11 TFLOPS effectifs. Il est limité par les lancements de noyaux et par Python.

Trois voies (gains = hypothèses)

1. PyTorch outillé (quelques jours).

- Contenu : `channels_last`, CUDA graphs pour le decoder et le base en batch 16, capture de toute la boucle coarse, Inductor pour fusionner les épilogues.

- VRAM : le pic PyTorch alloué historique est de 1,85 Go ; vérifier mémoire libre, réservée et contention avant toute capture.

- Contrainte : `torch.compile` est refusé par la politique actuelle (`terrain_inference.py:414-415`), pas par impossibilité technique.

- Gain attendu : 1,2 à 1,6× sur le NN.

2. TensorRT BF16 (1 à 3 semaines).

- Les formes statiques s'y prêtent bien.

- Gain attendu : decoder 1,5 à 2,5×, base 1,3 à 1,8×.

- Risques : la sélection de tactiques varie d'un build à l'autre ; il faudra figer le plan dans l'identité du monde et valider les motifs MP.

3. Moteur dédié CUTLASS/Triton (2 à 4 mois-personne).

- Contenu : poids pré-normalisés en NHWC, SiLU et upsample fusionnés en prologue, `mp_sum` et `clip` fusionnés en épilogue, mégakernel pour le coarse.

- Gain probable : seulement +10 à 30 % au-delà de TensorRT. Plafond NN ≈2 à 2,5×.

L'apport propre d'un moteur dédié

L'atout n'est pas la vitesse brute mais l'**invariance au batch**.

- **Aujourd'hui** : la composition d'un batch change les mètres (`terrain_window_scheduler.py:64-66`). C'est pourquoi coarse et decoder restent imposés en batch 1 (`terrain_inference.py:405-408`).

- **Avec des réductions à ordre fixe** : une invariance au batch serait à vérifier sur toutes les opérations et kernels. Si démontrée, elle faciliterait :

- le batching coarse, alors que la préparation planétaire représente ≈6 160 fenêtres × 20 pas, soit ≈3-4 min GPU déduites ;
- le batching entre tuiles ;
- un mode canonique sans surcoût.

Coûts et limites

- **Pas de raccourci de précision sûr** : Ampere n'a pas de FP8. Doubler le pic passe par FP16 à accumulation FP16, INT8 ou la parcimonie 2:4. Ce sont des approximations, à garder en options séparées.

- **Égalité octet par octet** : un changement de noyau peut la rompre ; il doit être validé. Elle n'était déjà attestée que pour une carte, un pilote et un build Torch.

- Mesures avant de construire :

- profil `nsys/ncu` d'une empreinte LOD 3 et d'une LOD 2 ;
- compteur de FLOP ;
- microbenchmark des convolutions aux formes exactes ;
- prototype TensorRT du decoder seul.

- **Seuil de décision** : si le base atteint déjà plus de 60 % du débit atteignable, un moteur dédié ne se justifie que pour le déterminisme.

Optimisations ciblées

Tous les gains ci-dessous sont des hypothèses. Ils ne se multiplient pas : chaque levier retire sa part d'un même budget.

Travail généré

- Surcalcul des étapes intermédiaires.

- Cause : `index.html:91` demande la tuile parente entière au niveau de raffinement (`terrain_lod.js:48-57`). Une tuile LOD 2 couvre 31 km et une LOD 3 61 km, pour une vue native d'environ 27 × 20 km.
- Mesure : les 40 fenêtres decoder couvrent ≈61 × 61 km.
- Besoin réel déduit : 12 à 20 fenêtres decoder et ≈150 à 200 fenêtres base, au lieu de 374.
- Options : limiter LOD 3 aux latents dont le decoder visible a besoin, produire LOD 2 et LOD 1 comme mips du natif visible (`terrain_final_mips.py`), et sauter l'étape LOD 1, qui ne fait aucun forward mais coûte un aller-retour.
- Gain attendu : NN du temps-au-natif ≈5,5 → 2,2-2,6 s.
- Risques : il faut un protocole de tuiles découpées, et la composition des batches change, dont la fidélité et le déterminisme devront être revalidés, sans présumer la tolérance.
- **Boucle coarse** (`terrain_inference.py:177-187`) : 20 pas Python par fenêtre. Capturer le solveur entier, avec validation `torch.equal` comme dans `terrain_cuda_graphs.py:84`. Hypothèse : LOD 4 0,28 → 0,12-0,18 s, exactitude à valider.
- **Graph decoder** : aujourd'hui seuls coarse et base sont enveloppés (`terrain_inference.py:422-426`). Hypothèse de gain de 0 à 10 %, exactitude à valider, car le decoder est déjà limité par le GPU (1,18 s de CPU contre 1,83 s CUDA).

Maths exactes

- **Rééchantillonnage** : remplacer les convolutions unitaires (`terrain_nn_constants.py:71-81`) par `::2, ::2]` et une répétition nearest. Multiplier un terme unique par 1 ne change aucun bit. Gain : 2 à 5 %.
- **mp_sum** (`terrain_nn_constants.py:49`) : supprimer le stack en conservant l'ordre FP32 puis l'arrondi. L'exactitude reste à prouver. Gain : 3 à 8 %.
- **Bruit portable** (`terrain_inference.py:302-304, 338-340`) : le préparer sur CPU pendant le forward précédent. Gain ≤5 %. Pas de RNG GPU sans preuve bit à bit.

Pipeline et transport

- **Aperçus CPU dans la voie GPU** : `terrain_server.py:774-786` prend `gpu_lock` même au

LOD ≥7, sans NN. L'aperçu mondial (`terrain_server.py:949-958`, priorité 0) occupe la voie ≈1,4 s, d'après l'écart entre réponses. Créer une voie CPU séparée. Hypothèse de gain : 1 à 2 s sur un monde froid ; pas d'attribution causale mesurée.

- **Finalisation** : trois écritures atomiques puis une relecture `np.load` (`terrain_server.py:804-836`). Le sémaphore `cpu_slots` (`terrain_jobs.py:60, 173`) peut bloquer la voie de calcul ; les compteurs donnent ≈60 ms d'encodage par job en moyenne. Servir depuis la mémoire et persister en différé. Gain : 0,5 à 1,5 s sur la phase native.
- **Client** : quatre transferts au plus (`index.html:103`), une requête par tuile, temporisations de 50 et 250 ms (`index.html:80, 109`). Un endpoint par vue ou par lot gagnerait 0,3 à 1 s.
- **Démarrage** : précharger les modèles, aujourd'hui chargés au premier accès (`terrain_server.py:175-178`, 2,5 à 3,6 s).
- **Petits coûts par requête, à mettre en cache** :
 - `world_identity` recalcule le SHA du manifest (`terrain_manifest.py:157-166`, appelé depuis `terrain_server.py:677` et 509) ;
 - `existing_final_mip` relit jusqu'à ≈36 fichiers (`terrain_server.py:462-481`) ;
 - le climat recalcule sa base de 15 cellules à chaque tuile (`terrain_climate.py:38-41`).
- **Latence perçue** : préparer les voisins quand le GPU est inactif rend les revisites rapides (≈0,5 s). Ce n'est pas un gain du générateur.

Hors goulot

- **Rendu WebGPU** : 0,4 ms historique par soumission CPU, sans chronométrage GPU ni présentation (`terrain_renderer.js:180-206`).
- **Transport localhost** : 391 Ko par tuile.
- **Bootstrap** : unique par seed. On peut gagner 0,3 à 0,6 s sur la copie et le SHA du binaire, `savez_compressed` et `fsync` (`terrain_bootstrap.py:148-157, 298-308`).
- **Cache coarse** : SHA complet à chaque chargement (`terrain_coarse.py:384-402`). Acceptable.

Plan d'expériences et décision

Fixer la métrique x5

- **Métrique** : le temps froid jusqu'à la couverture native d'une vue de 900 × 650 px, avec poids résidents, sans préparation ni préchargement.
- **Repère historique naturel** : 7,57 s, donc cible illustrative $\leq 1,5$ s. Pour le repère terrestre de 12,44 s : $\leq 2,49$ s. Refaire la baseline actuelle.
- **Mesures séparées** : premier aperçu utile, débit NN en km² par seconde, empreinte LOD 3 et LOD 2.
- Le x5 par empreinte n'est pas étayé (page 1) : valider le coût réel avant de conclure.

Expériences priorisées

1. E0 - Instrumentation (2 à 3 jours).

- Contenu : timeline serveur par job (file, verrou, CPU, forwards par modèle, encodage, écriture) et timeline client. nsys et ncu sur une LOD 3 et une LOD 2, plus compteur de FLOP et microbenchmark des convolutions.
- Go si le hors-NN dépasse 1,5 s, ou si le decoder reste sous 40 % du débit atteignable.

2. E1 - Pipeline exact (3 à 5 jours).

- Contenu : voie CPU séparée, écriture différée, saut de LOD 1, préchargement des modèles, caches d'identité.
- Validation : --require-exact.
- Attendu : 7,57 → 5,5-6,5 s.

3. E2 - Politique d'étapes (1 semaine).

- Contenu : intermédiaires limités à la vue, LOD 2 produit comme mip du natif.
- Go si le nombre de fenêtres decoder baisse au moins de moitié.

4. E3 - Exact côté NN (1 semaine).

- Contenu : capture du solveur coarse, rééchantillonnage exact, graph decoder, préparation du bruit.
- Validation : égalité octet par octet.

5. E4 - Decoder non exact (1 à 2 semaines).

- Contenu : channels_last avec compile, puis TensorRT BF16.
- Go si le gain atteint au moins 1,5× sous la porte de tolérance.

6. E5 - Moteur dédié (2 à 4 mois).

- Seulement si E0 et E4 laissent une marge, ou si l'invariance au batch devient un objectif produit.

Budget composé, sans multiplier les gains

- **Après E2-E4** : base $\approx 0,65$ -0,8 s, decoder $\approx 0,5$ -0,8 s, coarse $\approx 0,15$ s. NN $\approx 1,2$ -1,8 s.

- **Après E1** : hors-NN $\approx 0,4$ -1,0 s, en partie recouvrable par le calcul GPU.

- **Total** : $\approx 1,6$ -2,8 s, soit $\approx 2,7$ à 4,7×.

- **x5** : atteint seulement avec un recouvrement CPU/GPU, le haut de chaque fourchette, ou E5.

- **Protocole de décision** : 5 seeds × 3 sites, au moins 10 répétitions, horloges et charge GPU journalisées.

- **Critère final** : médiane $\leq 1,5$ s et p90 ≤ 2 s, sans dégrader le premier aperçu.

Fidélité et reproductibilité

- **Niveau 1, changements exacts** : égalité octet par octet de la hauteur et des cinq climats, zéros signés compris (benchmark_runtime_lod.py:126-148).

- **Niveau 2, changements de noyaux** :

- porte ≤ 1 m et limites climatiques (benchmark_inference.py:128-132) ;
- plus histogrammes de pente, part de côtes, spectre et raccords, sur un corpus élargi ;
- nouvelle identité runtime, déjà prévue par terrain_server.py:58-72 ;
- plan TensorRT et timing cache figés ;
- anciens caches jamais réutilisés, PyTorch conservé comme référence.

- **Niveau 3, approximations** : options nommées, jamais par défaut. Candidates :

- onestep_latent, ≈ -45 % de travail base ;
- moins de pas coarse ;
- accumulation FP16 ou INT8 ;
- distillation.

Les évaluer avec terrain_diffusion/evaluation/.

- **Test d'invariance au batch** : même fenêtre en batch 1 et en batch 16. Aujourd'hui, l'ordre de navigation peut déjà changer les octets dans la tolérance ; canonical_latents existe pour l'éviter.

Coûts

- E0 à E3 : 2 à 3 semaines-personne.
- E4 : 1 à 2 semaines.
- E5 : 2 à 4 mois, plus la maintenance par GPU et par pilote.

Avis

- **Générateur seul** : x5 sans approximation paraît peu plausible selon ces estimations. $\approx 2\times$ est une hypothèse de gain, pas un plafond démontré.
- **x5 perçu, du froid jusqu'au natif** : plausible mais non garanti. Il viendra d'abord du fait de calculer moins et du pipeline, ensuite d'un compilateur.
- **Moteur complet** : ne rien construire avant les mesures d'E0 et d'E4.